Reviewer Pack — Coxeter Inversion Spread of Barrington BPs Bounds NC^1 Depth

Entry 66b589aa0f3d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 04:47:50 UTC

Coxeter Inversion Spread of Barrington BPs Bounds NC^1 Depth

Entry ID: `66b589aa0f3d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-18 04:47:50 UTC

1. Conjecture

Field A (mathematical branch): Coxeter-group combinatorics of S_5 — the inversion-count / Coxeter length □_C: S_5 → {0,1,...,10} relative to the simple-transposition generators s_i=(i,i+1), in the Bjorner–Brenti / Humphreys finite-Coxeter-systems tradition (reflection length, weak order, Eulerian-statistics). This corner of finite Coxeter combinatorics is essentially absent from NC^1 / Barrington literature: an arXiv search 'Coxeter length' AND ('Barrington' OR 'branching program' OR 'NC^1' OR 'formula size') returns 0 direct hits and <5 adjacent papers, none using inversion-count statistics on BP partial-product traces. Distinct from the blacklisted character-mixing attempt: that invariant was a Plancherel/cycle-type Fourier functional, while □_C is a NON-ring, geometric word-length on S_5 (so it does not extend to polynomial relaxations and cannot algebrize). **Field B** (complexity object): NC^1 formula leaf size via Barrington 1989 width-5 permutation BPs over S_5: every depth-d Boolean formula compiles into a length-4^d width-5 BP whose terminal permutation is e iff f(x)=0 and a fixed 5-cycle σ iff f(x)=1. The structural form of the invariant (a property of the BP wiring, not of the truth table) shields it from the Razborov–Rudich natural-proofs barrier, since two BPs with identical truth tables can yield different σ² values.

Statement:

> Let B be a width-5 Barrington BP of length L on n Boolean inputs, with layers (i_k, g_k^0, g_k^1)*{k=1..L}; for x ∈ {0,1}^n let π_k(x) := g_1^{x*{i_1}} · g_2^{x_{i_2}} · ... · g_k^{x_{i_k}} ∈ S_5 and let □_C(π) be the inversion count of π. Define σ²(B) := E_{x~U({0,1}^n)} [(1/L)·Σ_{k=1}^L (□_C(π_k(x)) − μ_x)²], where μ_x := (1/L)·Σ_k □_C(π_k(x)). Conjecture: (i) for every BP B obtained by Barrington-compiling a depth-d AND/OR/NOT formula via the canonical Ben-Or–Cleve construction (so L = 4^d), σ²(B) ≤ C_1 · d for an absolute constant C_1 ≤ 4; (ii) for every non-constant width-5 BP B (Barrington-compiled or arbitrary), σ²(B) ≥ c_2 > 0 with c_2 ≥ 1/8. A single BP violating either bound refutes.

Rationale (proposer's reasoning):

> Coxeter length is a quantitative geometric measure of how far the partial-product walk wanders inside the Cayley graph Cay(S_5, simple transpositions); Barrington-compiled BPs traverse highly structured commutator-stamped excursions whose inversion-trace has tightly-controlled fluctuations governed by the recursion depth, while any non-trivial BP must achieve at least one □_C > 0 step and thus a nonzero variance. If true, σ² provides

a depth-sensitive, truth-table-blind certificate that lower-bounds the formula depth realising a given BP — a structural NC^1 separator immune to natural-proofs and algebrization, since □_C is not a polynomial functional and does not extend to any commutative-ring relaxation of the Boolean cube.

Taxonomy category: `BARRINGTON_ALG` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6d79984b2bd3c6ce`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 30 seeds × 4 depths (120 Barrington-compiled BPs) plus 2 hand-built adversarial probes, every BP must satisfy $\sigma^2(B) \leq 4 \cdot d$ (clause i) AND $\sigma^2(B) \geq 1/8$ (clause ii). Support requires 100% compliance on both bounds across all 122 BPs; any single violation falsifies.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Barrington branching program width-5 permutation S_5 invariant - Coxeter length inversion statistics random walk symmetric group word - permutation branching program NC1 lower bound formula size variance`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Constants
C1 = 4
c2 = Fraction(1, 8)

# Helper functions
def permutation_multiply(p1, p2):
    return tuple(p1[p2[i]] for i in range(len(p1)))
```

```
def barrington_compile(d):
    sigma = (1, 2, 3, 4, 5)
    layers = [sigma]
    for _ in range(4**d - 1):
        layers.append(permutation_multiply(sigma, sigma))
    return layers

def compute_sigma_squared(layers, n):
    L = len(layers)
    mu_x = [0] * (2**n)
    for x in range(2**n):
        pi = layers[0]
        for k in range(1, L):
            pi = permutation_multiply(pi, layers[k][x & 1])
        mu_x[x] = sum(layers[k].count(pi) for k in range(L)) / L
    sigma_squared = 0
    for x in range(2**n):
        pi = layers[0]
        for k in range(1, L):
            pi = permutation_multiply(pi, layers[k][x & 1])
            sigma_squared += (1/L) * ((layers[k].count(pi) - mu_x[x]) ** 2)
    return sigma_squared

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [2**d for d in range(1, 5)]
    results = []

    for n in n_values:
        layers = barrington_compile(int(n**(1/4)))
        L = len(layers)
        sigma_squared = compute_sigma_squared(layers, n)

        results.append({
            "n": n,
            "L": L,
            "sigma_squared": sigma_squared
        })

    mean_sigma_squared = sum(result["sigma_squared"] for result in results) / len(results)
    std_deviation = math.sqrt(sum((result["sigma_squared"] - mean_sigma_squared) ** 2 for result i

    conjecture_holds = all(sigma_squared <= C1 * d and sigma_squared >= c2 for n, L, sigma_squared
    counterexample = "" if conjecture_holds else "sigma_squared out of bounds"

    return {
        "metric_name": "sigma_squared",
        "metric_value": mean_sigma_squared,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
```

```
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dee058bd.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dee058bd.py", line 55, in run_trial
    layers = barrington_compile(int(n**(1/4)))
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dee058bd.py", line 30, in barrington_co
    layers.append(permutation_multiply(sigma, sigma))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dee058bd.py", line 24, in permutation_m
    return tuple(p1[p2[i]] for i in range(len(p1)))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dee058bd.py", line 24, in <genexpr>
    return tuple(p1[p2[i]] for i in range(len(p1)))
                 ~~^^^^^^^
IndexError: tuple index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an IndexError in permutation_multiply before any σ²(B) data was produced, so neither the support nor the falsification condition can be evaluated. | next: Fix the Barrington compiler bug (permutation_multiply receiving a malformed tuple from barrington_compile) and rerun the 30×4 + 2 probe protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	218051
2	preregistration	claude_max	opus	0	0	5700
3	novelty	claude_max	opus	0	0	4160
4	novelty	claude_max	opus	0	0	9190
5	test_gen	mistral	codestral-latest	0	0	312650
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	252045
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	247345
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	202827
9	judge	claude_max	opus	0	0	5215

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1257182 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 1, 'ollama_remote': 3}

(full prompt+response transcripts available in `research/audit/66b589aa0f3d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/66b589aa0f3d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/66b589aa0f3d.tar.gz` (if generated)